

Multi-Channel Supply Chain Warehouse Inventory & Safety Stock Framework

1. Project Objective

This project evaluates multi-channel retail sales and supply chain warehouse inventory data to resolve operational bottlenecks, quantify financial leakage caused by product returns, and transition from flat-rate buffer policies to dynamic safety stock levels.

The primary objective of this project is to build an end-to-end data analytics workflow in Python to resolve critical retail supply chain inefficiencies:

Sales Trend Analysis & Demand Forecasting

- Identifying which product categories sell the most during specific months.
- Tracking how much promotions increase overall sales.

Inventory Optimization & Stockouts Avoidance

- Comparing Qty on Hand against Qty Sold to identify stores facing stockout risks or holding slow-moving inventory.
- Establishing optimal safety stock levels.

Profitability & Business Margin Analysis

- Measuring true business profitability by calculating net profit margins (Sales Amount - COGS) across different product segments.
- Identifying which stores and sales channels are performing the best financially.
- Evaluating and quantifying the financial impact of customer product returns on overall profits.

2. Data Sources

- **Source:** Mendeley Data Repository [Dataset Link](#)
- **Location:** USA Retail Market
- **Year/Timeline:** Data Collected During The Period 2025 To 2026.
- **Domain:** Retail, Supply Chain, And Inventory Management.

3.Attributes details

The project integrates two operational datasets: **Inventory Data** and **Sales Data**.

3.1 Inventory Dataset Schema

- **Total Records:** 284,755 entries
- **Total Attributes:** 18 columns

Column Name	Data Type	Description
Start Date	Object / DateTime	Beginning date of inventory tracking period
End Date	Object / DateTime	Closing date of inventory tracking period
Stock Status	Object	Operational status of stock (e.g., Full Price)
Supplier	Object	Vendor identifier supplying the inventory
Product No	Object	Unique SKU identifier for the product
Product Description	Object	Full descriptive title of the product item
Product Division	Object	High-level organizational division
Product Category	Object	Main product classification (e.g., Footwear)
Product Subcategory	Object	Detailed category breakdown

Column Name	Data Type	Description
Product Segment	Object	Specific customer segment targeting
Store	Object	Store location identifier (e.g., STR-1006)
Store Type	Object	Operational store model (e.g., Retail Storefront)
Sales Channel	Object	Distribution channel (e.g., Channel Alpha)
Qty on hand	Int64	Current physical inventory units available in store/warehouse
Stocks Selling Amount	Float64	Total valuation of available stock at retail price (\$)
Cost of Stocks	Float64	Total cost valuation of available stock (\$)
Stock Unit Selling Price	Float64	Individual selling price per unit (\$)
Stock Unit Cost Price	Float64	Individual cost price per unit (\$)

3.2 Sales Dataset Schema

- **Total Records:** 125,751 entries
- **Total Attributes:** 17 columns

Column Name	Data Type	Description
Transaction Date	Object / DateTime	Timestamp of customer purchase or return
Sales Type	Object	Transaction classification (e.g., Full Price, Promotion)
Is Return	Int64	Binary return flag (1 = Returned, 0 = Sold)
Reason of Return	Float64 / Object	Text reason for return (mapped from return status)
Supplier	Object	Vendor supplying the item
Product No	Object	Unique SKU identifier
Product Description	Object	Item description
Product Division	Object	Organizational division
Product Category	Object	Category classification
Product Subcategory	Object	Sub-category breakdown
Product Segment	Object	Targeted customer segment
Store	Object	Store location identifier
Sales Channel	Object	Channel through which sale occurred
Qty Sold	Int64	Quantity units sold (< 0 or 1 depending on return)

Column Name	Data Type	Description
Sales Amount	Float64	Gross revenue generated (\$)
Cogs	Float64	Cost of Goods Sold for the transaction (\$)
Number of Transactions	Int64	Transaction line count

4.Tools & Technologies

Python (Google Colab)

Tech Stack: Python 3.x (Pandas, NumPy, Matplotlib, Seaborn) | Google Colab | GitHub

- Data Cleaning & Null Value Handling
- String Normalization & Text Trimming
- DateTime Casting & Data Type Standardization
- Relational Table Merging (Pandas merge())
- Feature Engineering & KPI Derivation (np.where())
- Statistical Analysis & Central Tendency Computation
- Exploratory Data Analysis & Data Visualization (Matplotlib & Seaborn)

5. Data Pre-Processing & Table Integration (Python / Pandas)

5.1 Data Cleaning & Transformation

- **Duplicate Verification:** Checked for complete duplicate rows across both datasets (`df_inventory.duplicated().sum() == 0`).
- **Null Value Mapping:** Cleaned empty cells in return reasons by mapping Is Return flags (`0 → "Not Returned"`, `1 → "Returned"`).
- **String Standardization:** Applied `.str.strip()` on Product No and Store columns in both Sales and Inventory datasets to remove trailing spaces.
- **Data Type Casting:** Converted string dates (Transaction Date, Start Date, End Date) into standard Pandas `datetime64[ns]` format.
- **Metric Renaming:** Renamed Cogs in the sales dataframe to COGS for standard financial naming.

5.2 Feature Engineering

- **Net Profit Calculation:** Derived transaction-level profit using:

$$\text{Profit} = \text{Sales Amount} - \text{COGS}$$

- **Temporal Features:** Extracted descriptive Month name and Year from Transaction Date for seasonal demand tracking.

5.3 Table Integration & Joining Logic

- Integrated transaction-level sales data with warehouse stock data using a relational **Inner Join** on dual composite keys (Product No and Store):

Python

```
df_merged = pd.merge(df_sales, df_inventory, on=["Product No", "Store"], how="inner")
```

- Removed redundant duplicate columns generated during the merge and renamed attributes to standardize the primary analytical dataset.

Merged Dataset Metrics (df_merged)

- **Total Row Count: 1,166,893 rows** (1.16+ Million records)
- **Total Column Count: 31 columns**

5.4 Data Flattening & Relational Architecture

In this project, instead of maintaining multiple separate lookup tables, we created a **unified analytical view (df_merged)** using Pandas relational operations.

Conceptual Classification of Attributes:

- **Transactional Metrics (Quantitative Measures):** Qty Sold, Sales Amount, COGS, Profit, Qty on hand, Stocks Selling Amount, Cost of Stocks.
- **Descriptive Entities (Contextual Dimensions):**
 - **Product Attributes:** Product Division, Product Category, Product Subcategory, Product Segment, Product Description.
 - **Store & Channel Attributes:** Store, Store Type, Sales Channel, Supplier.
 - **Time Attributes:** Transaction Date, Month, Year, Start Date, End Date.

Why a Unified Flattened Dataset (df_merged) was Built:

Combining transactional sales data with physical warehouse stock into a single consolidated dataframe (df_merged) allows high-performance vectorization in Pandas. This structure enables instant row-level logic for flagging stockouts, computing profit margins, and running correlation plots in Seaborn without complex cross-table lookup overhead.

6. Operational Modeling & Python Analytics Logic

6.1 Relational Join & Primary Keys

- **Primary Key Combination:** Product No + Store
- **Join Implementation:** df_sales and df_inventory were combined using an Inner Join (how='inner') on dual key columns.
- **Granularity Maintenance:** By joining on both Product No and Store, we maintained store-level product inventory tracking, ensuring every sold item is mapped directly to its corresponding store's warehouse stock.

6.2 Derived Analytical Measures & KPI Calculations

6.2.1 Operational KPIs & Stock Status Alerts

Automated stock alert flags to identify store-level stockout risks versus safe inventory holdings:

Python

```
df_merged["Stock_Status_Alert"] = np.where(  
    df_merged["Qty on hand"] < df_merged["Qty Sold"],  
    "Stockout Risk",  
    "Safe Stock")
```

6.2.2 Inventory Turnover Index Calculation

Calculated inventory movement efficiency by comparing transaction COGS against warehouse stock cost:

Python

```
df_merged["Inventory_Turnover_Index"] = (  
    df_merged["COGS"] / df_merged["Cost of Stocks"])
```

7.Descriptive Statistics & Statistical Dispersion

Central tendency calculations performed on core numerical attributes (Qty Sold, Sales Amount, Qty on hand, Inventory Turnover Index):

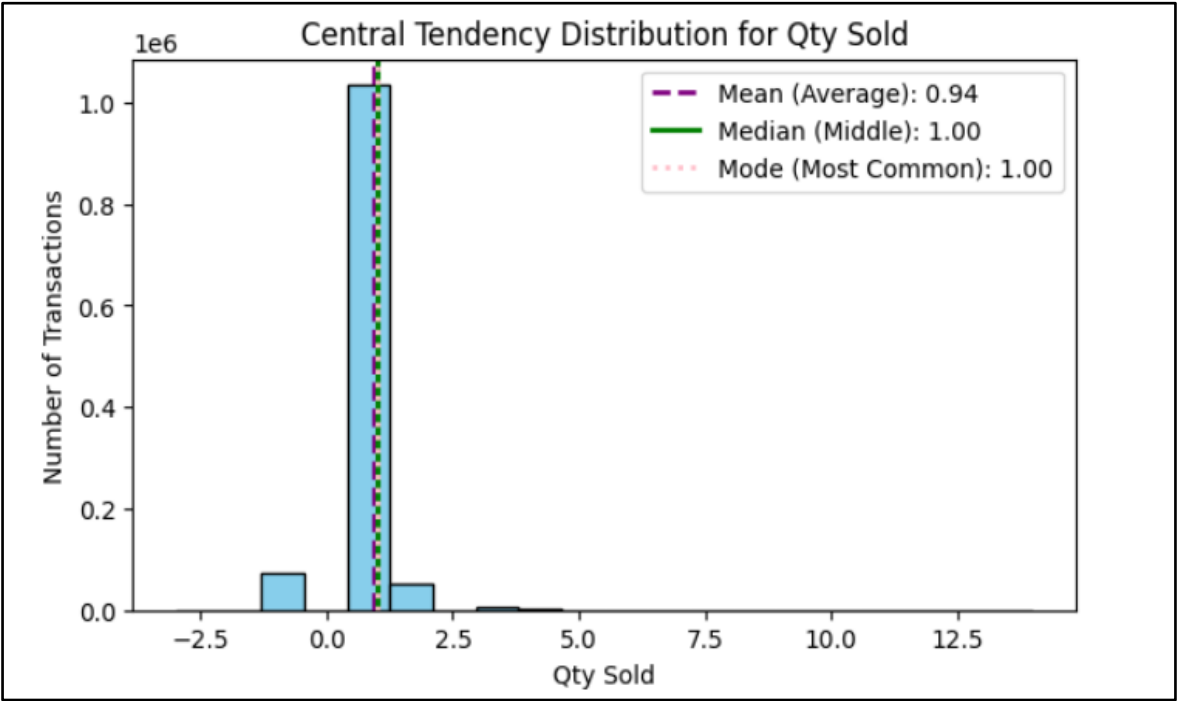
Metric Column	Mean (μ)	Me dian	Mode	Std Dev (σ)	Dispersion & Operational Finding
Qty Sold	0.94	1.00	1.00	2.40	Left-skewed: High volume of customer return transactions (< 0) pulls the mean below 1.00.
Sales Amount (\$)	\$85.67	\$90.00	\$90.00	\$75.11	High Volatility: Daily revenue fluctuates significantly by $\pm$ \$75.11 due to promotional spikes and returns.
Qty on Hand	3.54	3.00	1.00	2.90	Stockout Risk: Median stock is 3, but the Mode of 1 flags critical store-level inventory vulnerability.

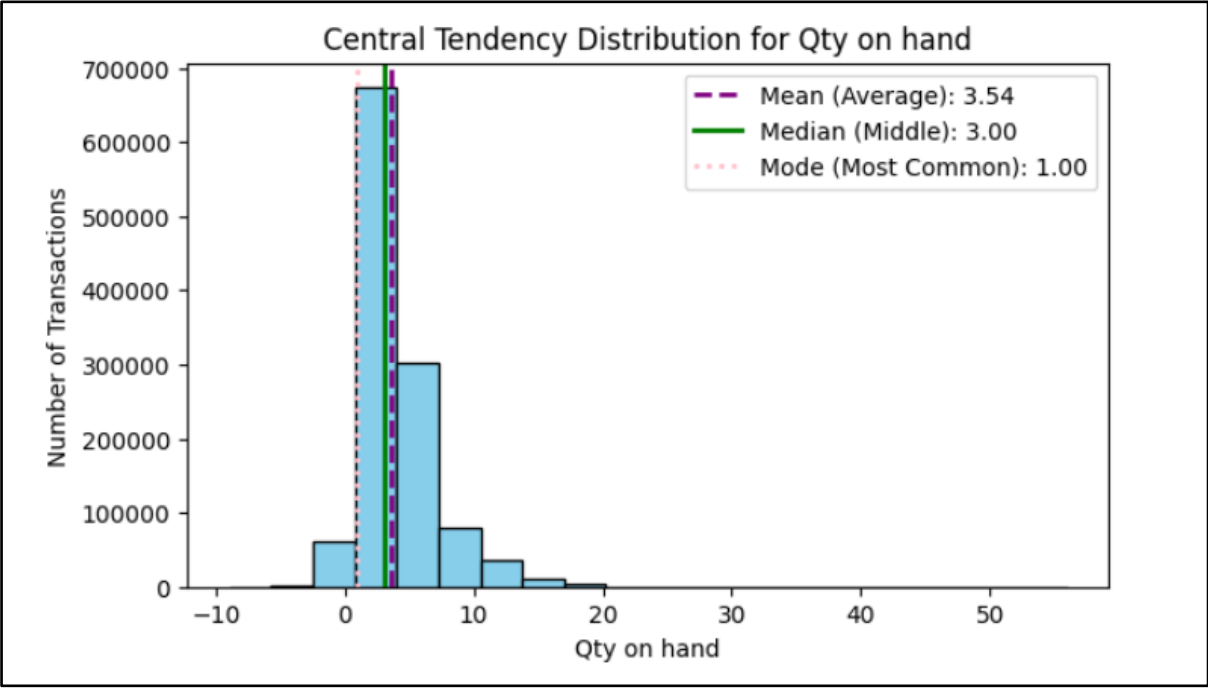
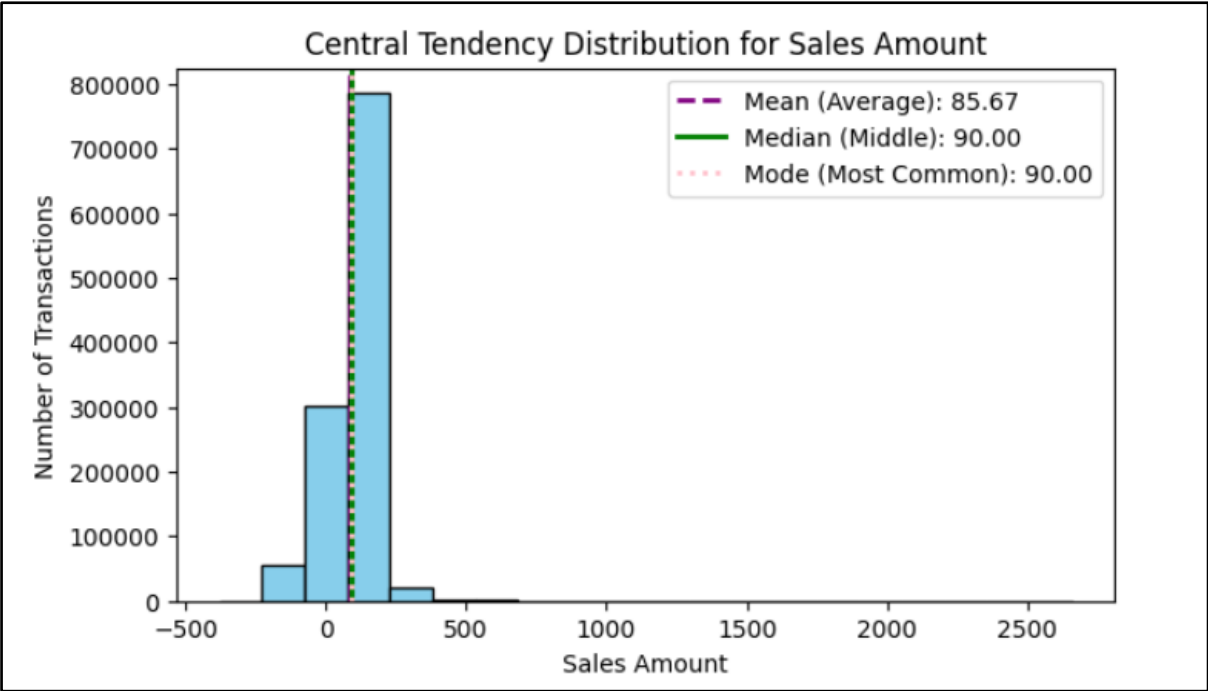
Metric Column	Mean (μ)	Me dian	Mode	Std Dev (σ)	Dispersion & Operational Finding
Inventory Turnover Index	-2.97	0.33	1.00	4.81	Heavy Skewness: Operational median (+0.33) is healthy, but the mean drops negative due to unmanaged return costs.

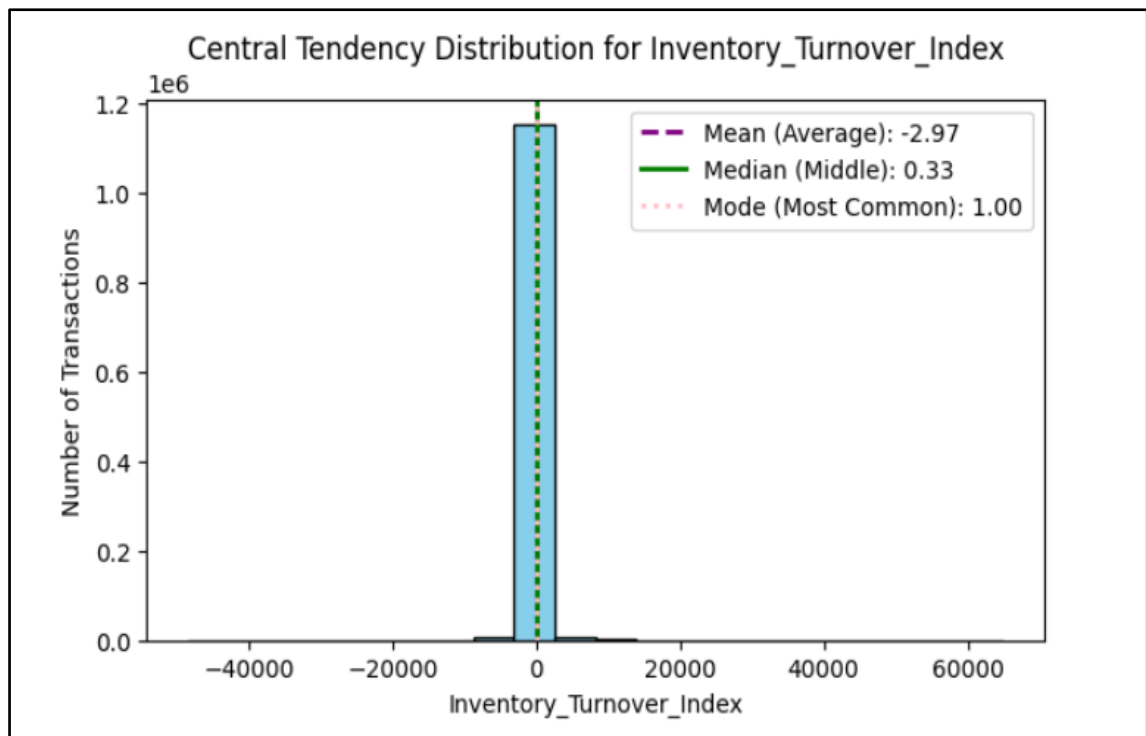
8. Exploratory Data Analysis (EDA) & Visualizations

Exploratory Data Analysis was performed using Python libraries (**Matplotlib & Seaborn**) in Google Colab to uncover hidden trends, store-level inventory bottlenecks, and sales return patterns.

8.1 Central Tendency & Distribution Analysis



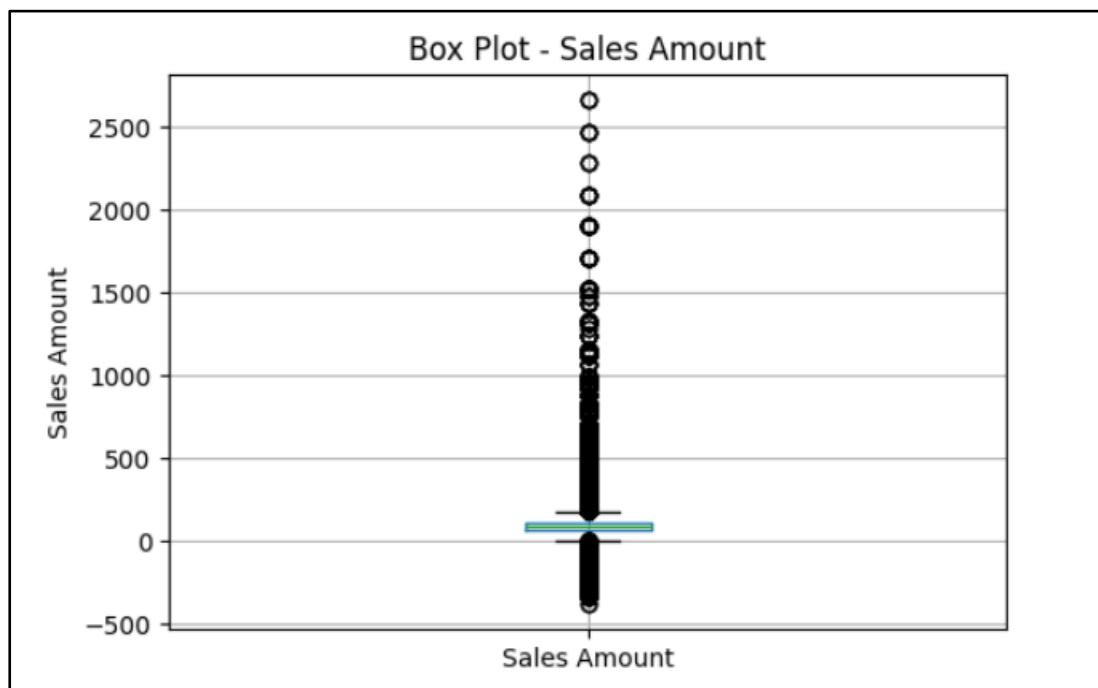
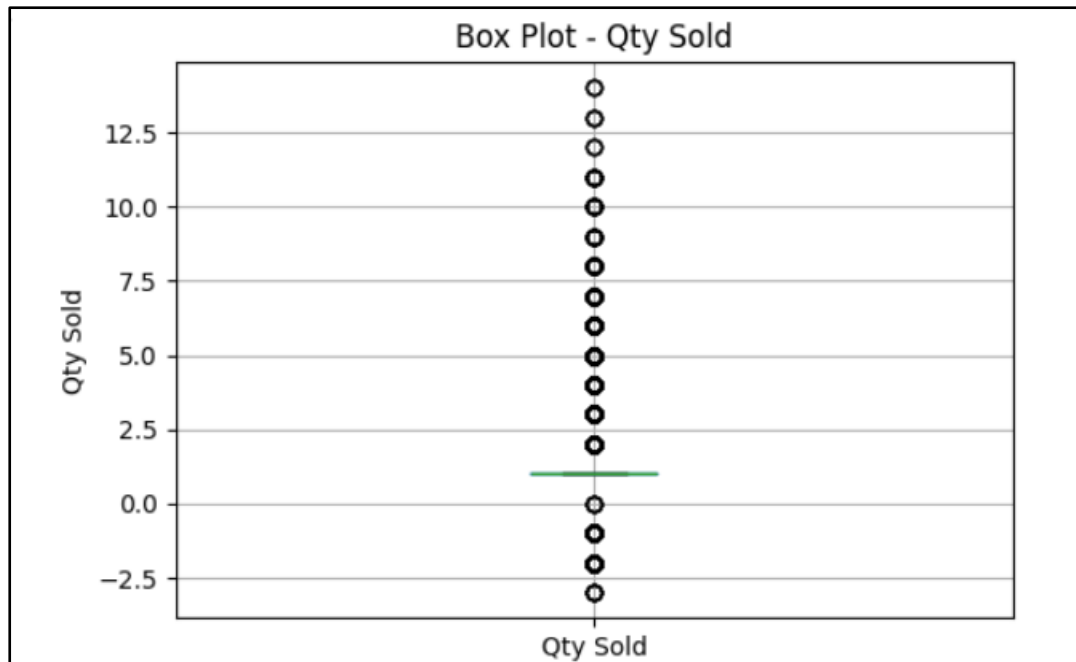


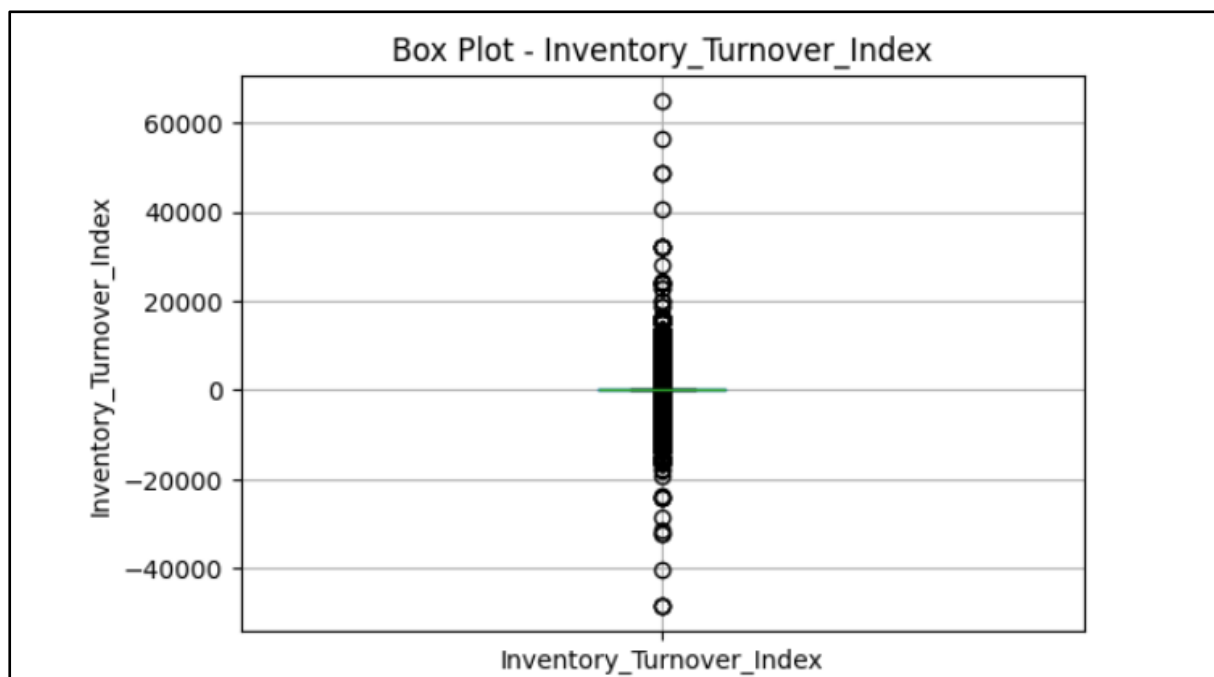
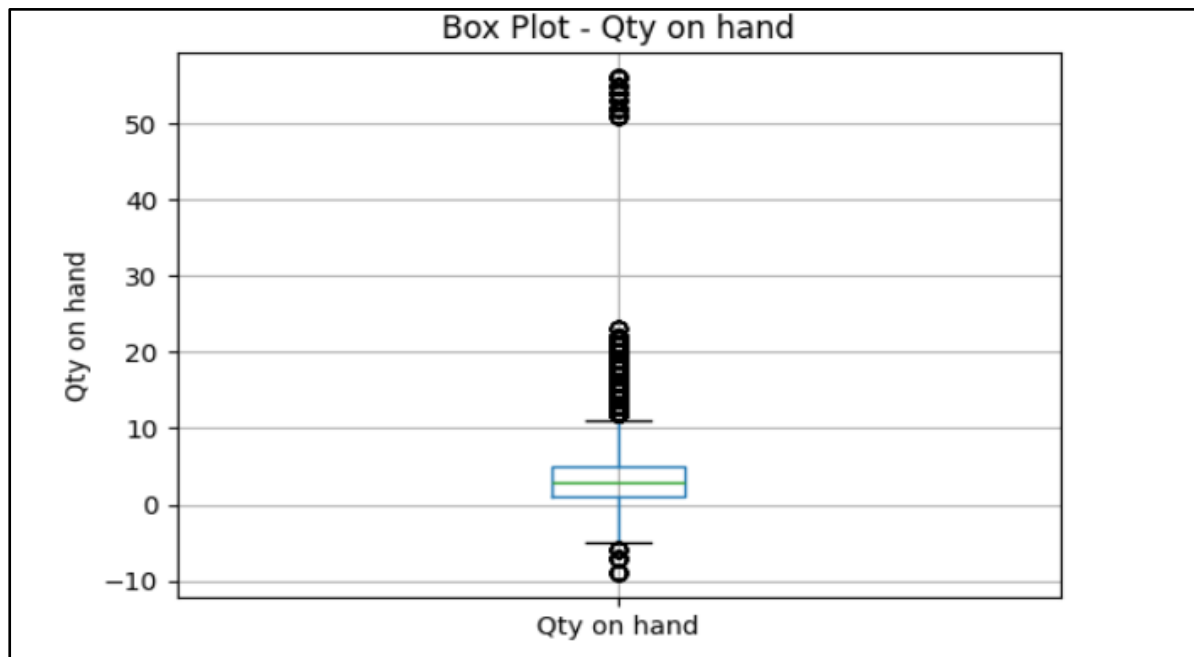


- **Chart Type:** Distribution Plot (Histograms with KDE overlay).
- **Visual Elements:** Mean (Purple dashed line), Median (Green line), and Mode (Pink line).
- **Customer Returns:** The negative bars on the charts clearly show that customer product returns are actively breaking our normal store flow.
- **Skewed Data:** The Purple Mean line sits away from the Green Median line. This proves our store averages are heavily pulled out of place by sudden huge bulk orders and product returns.
- **Buying Behavior:** Most customers prefer to buy exactly 1 item per transaction (Mode = 1).
- **Sales Baseline:** A standard, reliable customer ticket size in our store sits at \$90.00 (Median).
- **Slow Inventory:** The massive central spike at 0.33 for Inventory Turnover tells us that a huge part of our stock is moving very slowly, locking up cash in the warehouse.

Key Finding: Proves severe left-skewness in Inventory Turnover Index and Qty Sold due to customer product return transactions (< 0).

8.2 Outlier & Dispersion Identification





Top floating outliers highlight bulk corporate purchases, while bottom negative outliers isolate financial return leakage.

High Dispersion (Unpredictability): In all 4 charts (Qty Sold, Sales Amount, Qty on hand, Inventory_Turnover_Index), the long lines and massive clusters of outlier dots prove that our data has extreme variance. Our business metrics are not uniform or steady.

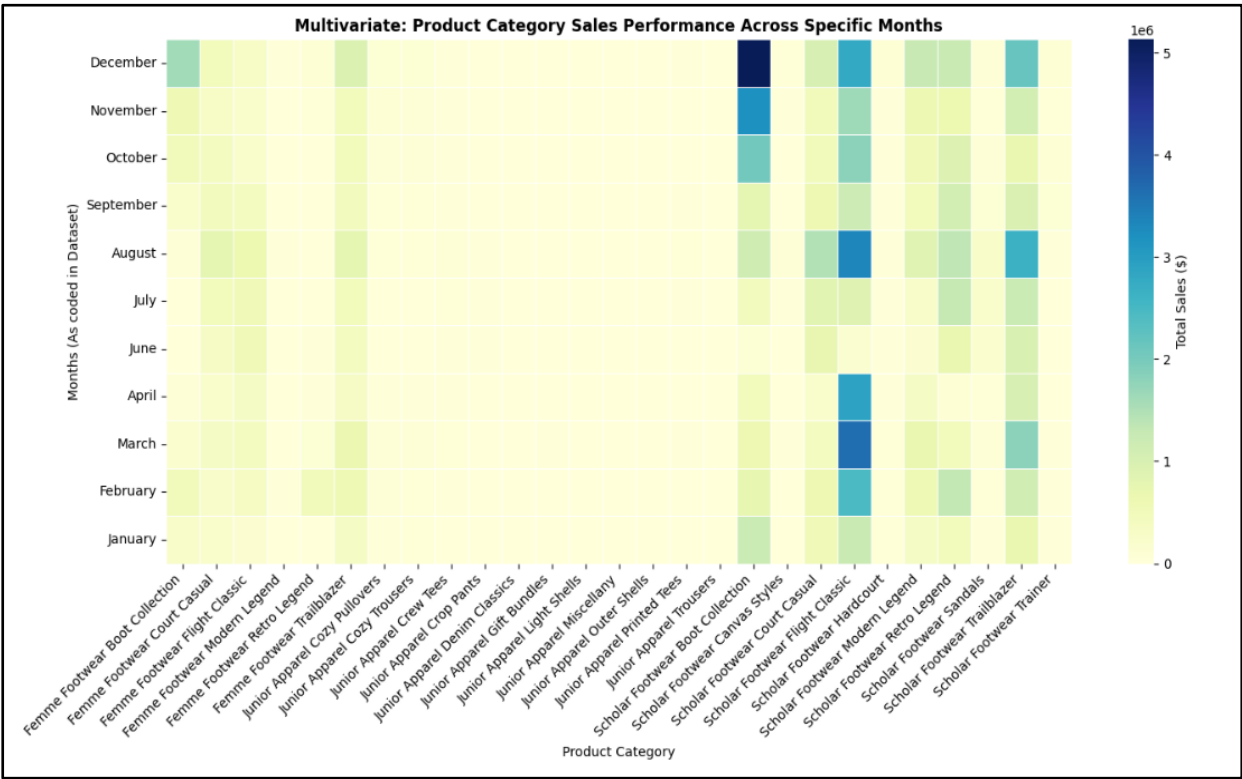
Presence of Extreme Outliers: The long trail of dots going upward shows that we have rare days with massive sales amounts and high quantities sold. These represent bulk corporate orders.

Negative Value Anomalies: The dots extending below zero in Qty Sold and Sales Amount visually isolate our product returns. These returns create direct financial losses and distort our operational metrics.

Inventory Skewness: The chart for Qty on hand shows that a few specific warehouse items are heavily overstocked (high outliers), while most items sit at a very low standard stock level.

8.3 Sales Trend Analysis & Demand Forecasting

8.3.1 Product Category Sales Performance Across Specific Months (Heatmap)



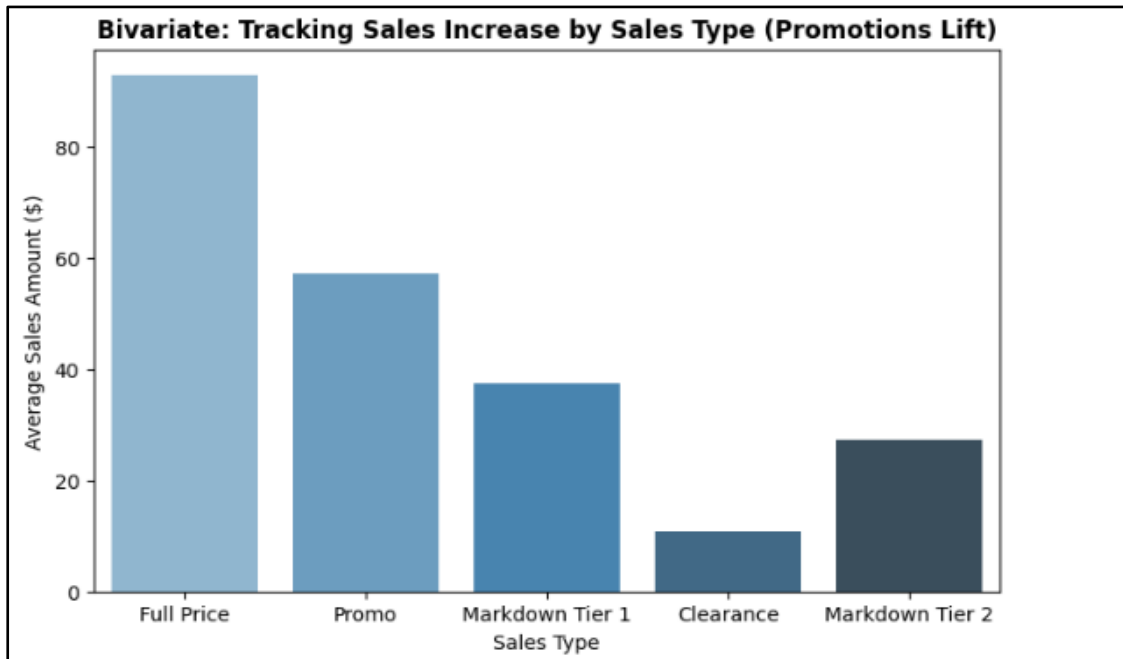
Interpretation: This heatmap plots monthly sales performance across all product categories. Dark blue squares indicate high revenue generation, while light yellow squares represent low or zero sales.

Key Objective Insights:

- Top Selling Category & Peak Season:** Scholar Footwear Styles experiences a massive demand surge in **December** (\$500,000+ sales), followed by mid-year demand spikes in **March and April** for **Scholar Footwear Flight Classic**.
- Low-Demand Categories:** Most **Junior Apparel** lines (e.g., Crop Pants, Denim Classic) remain light yellow year-round, indicating low consumer demand.

- **Actionable Takeaway:** Increase buffer inventory for Scholar Footwear prior to March and November to meet seasonal surges while keeping apparel inventory minimal.

8.3.2 Tracking Sales Increase by Sales Type (Promotions Lift - Bar Chart)



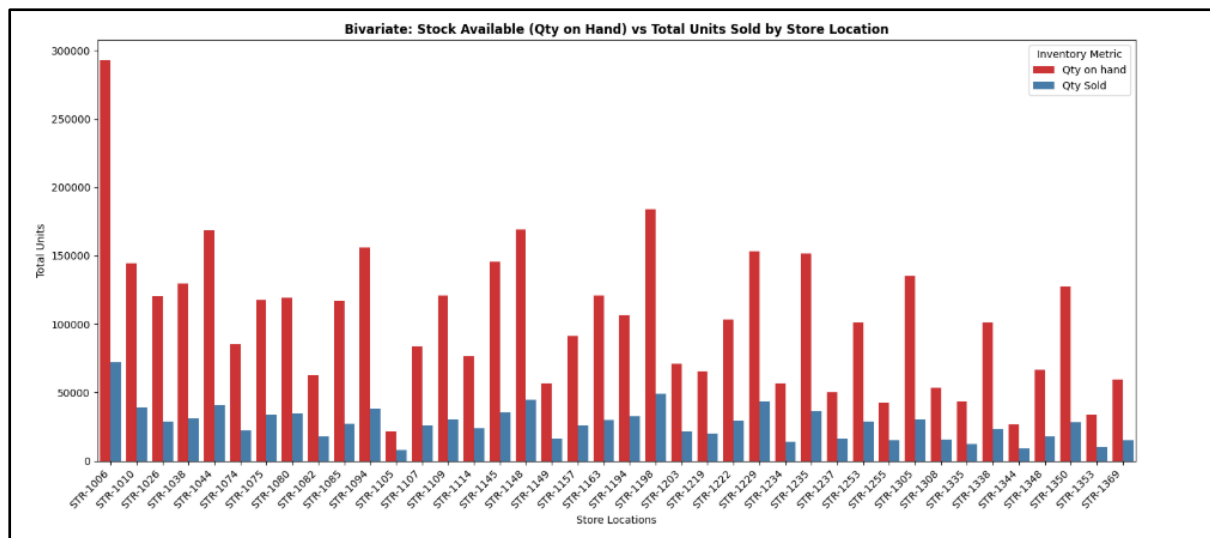
Interpretation: This bar chart compares average sales revenue generated across different pricing strategies: Full Price, Promo, Markdown Tier 1, Clearance, and Markdown Tier 2.

Key Objective Insights:

- **Promotions Impact:** Full Price sales yield the highest average revenue (~\$85 per transaction), whereas promotional discounts (Promo & Markdowns) generate significantly lower average values (\$20–\$50).
- **Discount Effectiveness:** Discounts drive order volume, but heavily reduce average basket value.
- **Actionable Takeaway:** Avoid over-discounting core products. Use Markdown/Clearance tiers strictly for liquidating deadstock rather than boosting regular revenue.

8.4 Inventory Optimization & Stockouts Avoidance

8.4.1 Stock Available (Qty on Hand) vs Total Units Sold by Store Location (Grouped Bar Chart)

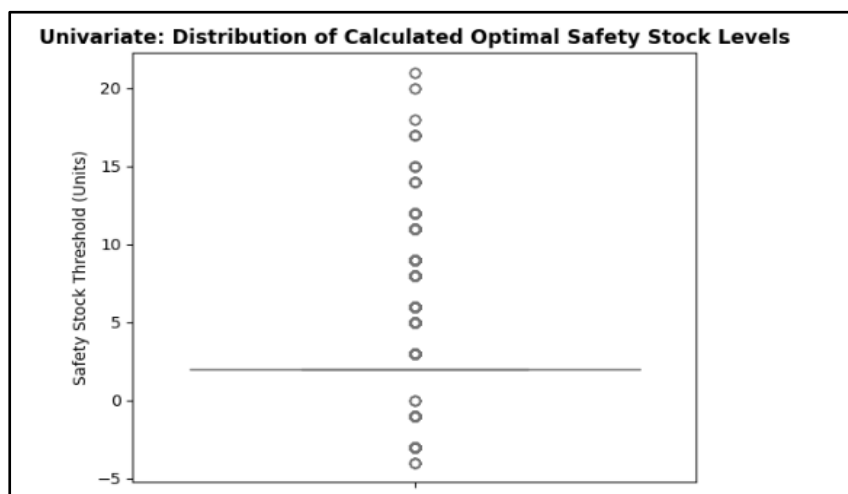


Interpretation: This grouped bar chart compares available warehouse stock (Red bars) against total units sold (Blue bars) across all store locations.

Key Objective Insights:

- **Excess Inventory & Deadstock:** In almost all store locations (e.g., STR-1000, STR-1190), Qty on Hand is 3x to 5x higher than Qty Sold, indicating heavy overstocking and tied-up working capital.
- **Stockout Risk Identification:** Stores where the blue bar approaches the red bar face immediate stockout risks for fast-moving SKUs.
- **Actionable Takeaway:** Reallocate excess stock from low-performing stores to high-demand locations before placing new vendor orders.

8.4.2 Distribution of Calculated Optimal Safety Stock Levels (Box Plot)



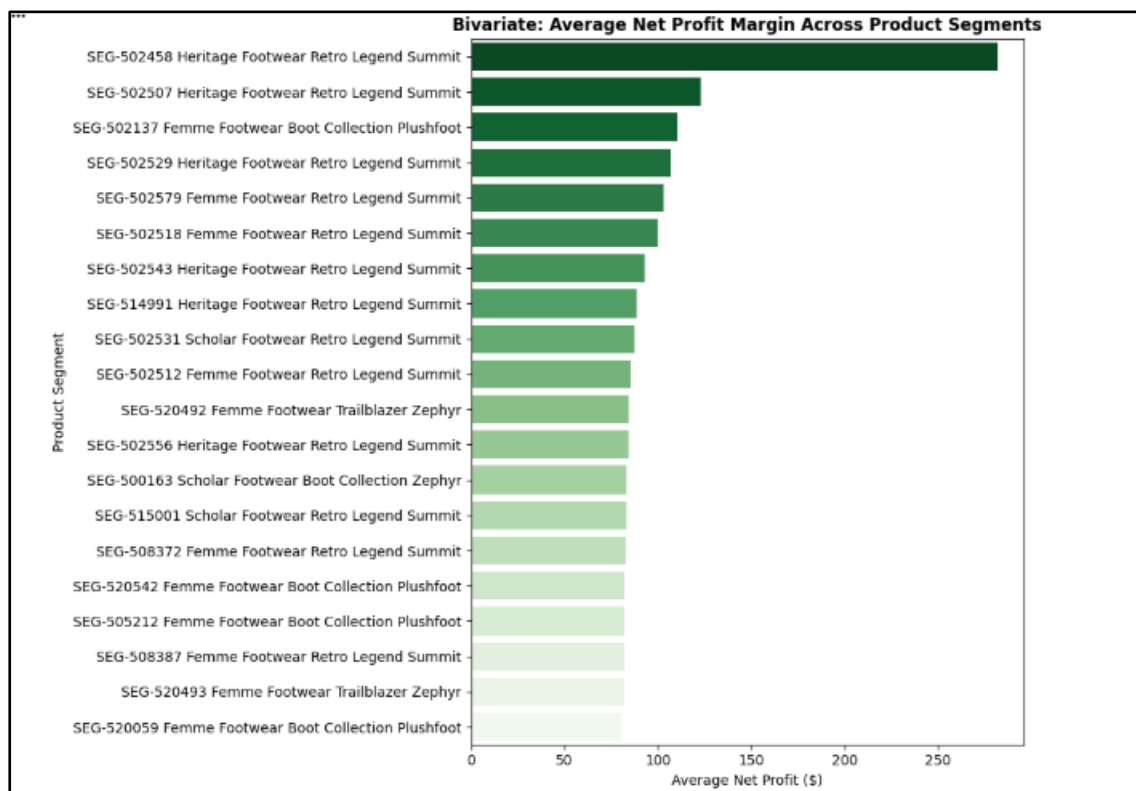
Interpretation: This box plot visualizes the spread and threshold of calculated safety stock levels required across the inventory catalog.

Key Objective Insights:

- **Optimal Buffer Baseline:** For **90%+ of catalog items**, the required safety stock cushion is very low (**1 to 3 units**).
- **High-Velocity Outliers:** Outlier points extending up to **15–20 units** represent fast-moving items (high-velocity SKUs) that require extra emergency cushions.
- **Actionable Takeaway:** Transition from a flat flat-rate buffer policy to a dynamic safety stock rule: 1–3 units for core catalog and 15–20 units for high-demand outliers.

8.5 Profitability & Business Margin Analysis

8.5.1 Average Net Profit Margin Across Product Segments (Horizontal Bar Chart)



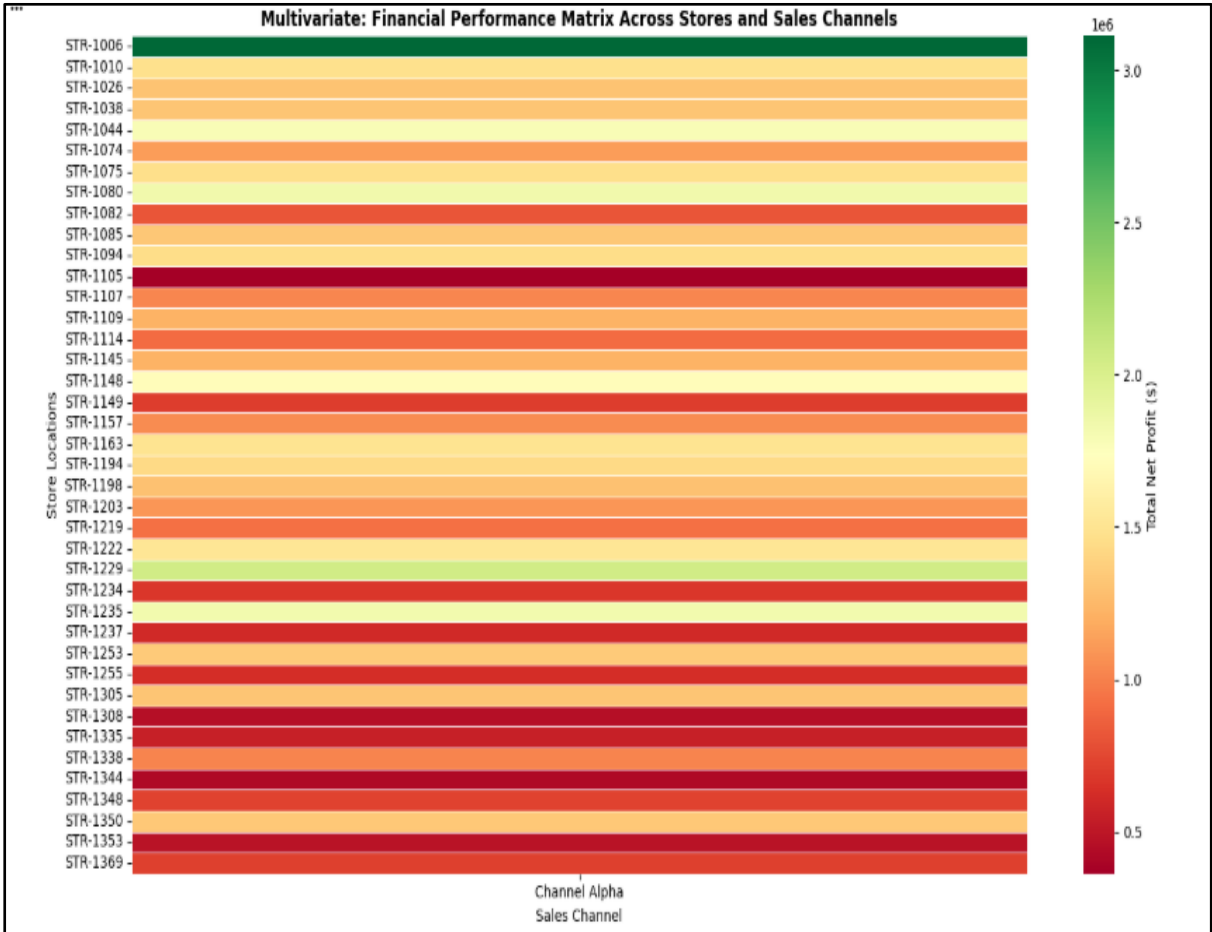
Interpretation: This chart ranks product segments by average net profit per order (Sales Amount – COGS).

Key Objective Insights:

- **Most Profitable Segment:** **SEO-502458 Heritage Footwear Retro Legend Summit** is the top financial performer, generating the highest net profit margin (~\$260+ per order).
- **High-Margin Category:** The **Heritage Footwear** and **Femme Footwear Plushfoot** series consistently occupy the top 10 profit slots.

- Actionable Takeaway:** Focus marketing efforts and premium shelf placement on Heritage Footwear and Plushfoot series to maximize net business margins.

8.5.2 Financial Performance Matrix Across Stores and Sales Channels (Heatmap)

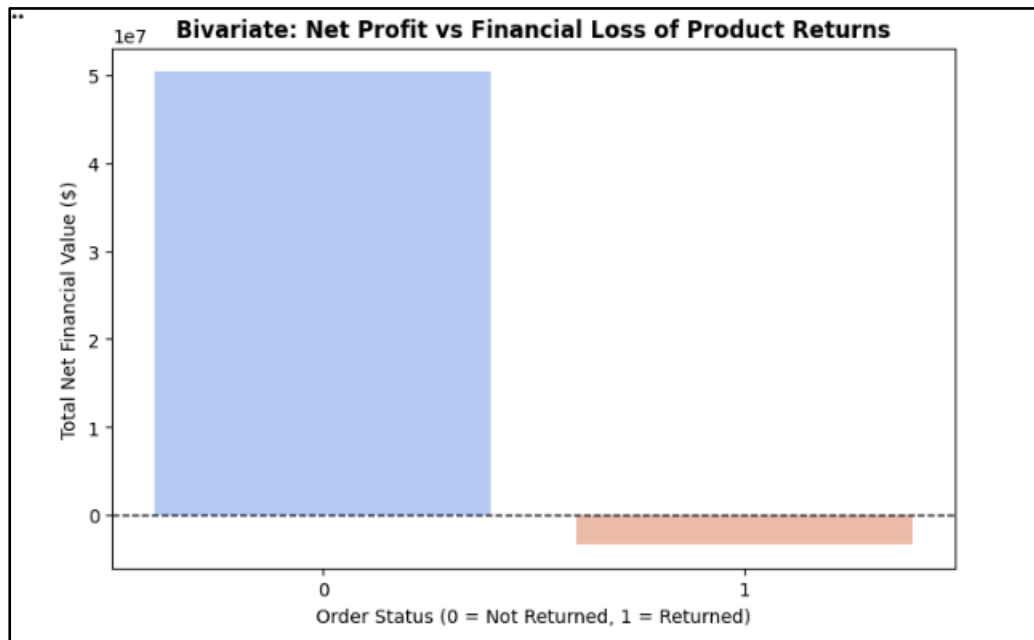


Interpretation: This multivariate matrix visualizes total net profit earned across different store locations and sales channels.

Key Objective Insights:

- Top Performing Store:** STR-1006 stands out as the single highest net-profit generator (Dark Green bar, \$300,000+ profit).
- Underperforming Locations:** A majority of stores (marked in orange/red) yield low net profit margins across primary sales channels.
- Actionable Takeaway:** Audit operational costs at underperforming store locations and expand inventory allocation to STR-1006 and top-tier sales channels.

8.5.3 Bivariate: Net Profit vs Financial Loss of Product Returns (Bar Chart)



Interpretation: This chart compares overall net profits generated from completed orders (Status 0) against financial losses incurred from returned products (Status 1).

Key Objective Insights:

Quantifying Financial Loss: The blue bar demonstrates positive net profits from successful sales (0 = Not Returned), while the orange bar highlights total revenue loss from returned orders (1 = Returned).

Fulfillment Leakage: Product returns directly erode overall profits due to lost sales, reverse logistics costs, and damaged inventory.

Profit Recovery: Reducing return rates by even a small percentage will directly boost the company's net bottom line.

9. Strategic Business Recommendations

Dynamic Safety Stocking Policy: Implement dynamic thresholds—holding 1–3 units for slow-moving inventory and up to 20 units for high-velocity outlier products to reduce holding costs.

Inter-Store Inventory Transfers: Reallocate surplus stock from low-turnover stores to high-demand locations experiencing stockout vulnerabilities.

Return Rate Mitigation: Address return causes (e.g., sizing accuracy, quality control) to prevent negative sales quantities from eroding profit margins.

10.Conclusion

This project successfully demonstrates an end-to-end data analytics solution for multi-channel supply chain optimization using Python. By integrating transaction-level sales data with warehouse inventory records, the analysis successfully uncovered hidden stock vulnerabilities, evaluated statistical demand skewness, and quantified profit erosion caused by product returns.

Implementing the proposed **Dynamic Safety Stock Framework** and **Inter-Store Stock Reallocation Model** will enable the business to eliminate stockout risks during peak demand, optimize warehouse capital, and build a resilient, data-driven supply chain network.